Classes, Models, and the Training Loop

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Explain what a Python class bundles together, and read `__init__`, `self`, `super()`, and `__call__` in code
- State the model interface used all semester: an `nn.Module` subclass with `__init__` (create parameters) and `forward` (inputs to predictions)
- Build a linear model, combine several into a multi-output model, and layer them into an MLP, all with the same interface
- Say which object does which job in a training loop, and write the loop from memory
- Recognize d2l's `training_step` / `loss` / `configure_optimizers` as optional packaging on top of that interface

Object-Oriented Python

Bundling data with behavior

Why classes?

Answering “are all models going to be classes?”

Why a model wants to be a class

- A model is really **two things**:
  - **Numbers it remembers** (the parameters W, b)
  - **A recipe that uses them** (the forward pass)

- **A class bundles both of these things**
  - A function alone forgets its numbers between calls

Anatomy of a class

```
 1 class Line:
 2     def __init__(self, w, b):
 3         self.w = w
 4         self.b = b
 5 
 6     def predict(self, x):
 7         return self.w * x + self.b
 8 
 9 line = Line(2, 1)
10 print(line.predict(3))
```

```
7
```

- `Line(2, 1)` builds an object; `line.predict(3)` calls a method on it
- The object **remembers** `w` and `b` between calls

Breakdown: `__init__`

```
1 def __init__(self, w, b):
2     self.w = w
3     self.b = b
```

- Runs automatically when we write `Line(2, 1)`: it is the **constructor**
- It sets up the object's **attributes**, the numbers the object remembers
- No field *declarations*: attributes exist the moment we assign to `self.something`

Breakdown: `self`

Java

```java
public double predict(double x) {
    return this.w * x + this.b;
}
```

- `this` is implicit

Python

```python
def predict(self, x):
    return self.w * x + self.b
```

- `self` is **written out** as the first parameter
- Python passes it for us:
  `line.predict(3)` becomes
  `Line.predict(line, 3)`

Java vs. Python: quick translation

	Java	Python
Constructor	same name as the class	`__init__`
Current object	`this` (implicit)	`self` (explicit first parameter)
Creating an object	`new Line(2, 1)`	`Line(2, 1)`
Fields	declared up front	created by assigning to `self.x`
Private	`private` keyword	convention only: `_name`

Dunder methods

- Names with ***d***ouble ***under***scores on both sides: `__init__`, `__call__`, `__len__`, …
- We never call them by name. **Python calls them for us** when we use certain syntax:

We write	Python runs
`Line(2, 1)`	`Line.__init__(...)`
`len(x)`	`x.__len__()`
`line(3)`	`line.__call__(3)`

`__call__`: objects that act like functions

```python
class Line:
    def __init__(self, w, b):
        self.w = w
        self.b = b

    def predict(self, x):
        return self.w * x + self.b

    def __call__(self, x):
        return self.predict(x)

line = Line(2, 1)
print(line(3))
```

```
7
```

- `line(3)` looks like a function call, but `line` is an object holding w and b
- **This is exactly why we will write `model(X)` for a model**

Inheritance and `super()`

```
1 class ReluLine(Line):
2     def __init__(self, w, b):
3         super().__init__(w, b)
4 
5     def __call__(self, x):
6         return max(0, super().__call__(x))
7 
8 print(Line(2, -10)(3), ReluLine(2, -10)(3))
```

```
-4 0
```

- `ReluLine(Line)` **is a** `Line`, and gets its methods for free
- `super()` reaches the parent's version: here we reuse `Line`'s setup and its `__call__`, then clip at zero
- `super().__init__(...)` is essential!

Important

Forgetting `super().__init__(...)` means the parent never sets up its part. With `nn.Module` this is a real bug, as we'll see.

Reference: Python classes

```
class Line:
    def __init__(self, w, b):        # constructor: remember the numbers
        self.w = w
        self.b = b

    def predict(self, x):            # a method: uses the remembered numbers
        return self.w * x + self.b

    def __call__(self, x):           # dunder: makes line(3) work
        return self.predict(x)


class ReluLine(Line):                # inheritance: a ReluLine IS a Line
    def __init__(self, w, b):
        super().__init__(w, b)       # let the parent set up its part

```

The model interface

What we expect of every model

What do we need from a model?

- Given an input X, produce predictions $\hat{y}$
- Hold **parameters** that training can change
- Be **easy to use**: move to a GPU, save it, switch it between training and evaluation
- Work with **any** optimizer and any loss without special cases

The Big Idea

A model is an object that owns its parameters and knows how to turn inputs into predictions. Everything else comes for free.

The minimal model

```python
import torch
from torch import nn

class Model(nn.Module):
    def __init__(self):
        super().__init__()          # gives us everything nn.Module can do
        # create parameters / sub-models here

    def forward(self, X):
        """Given X, return y_hat."""
        ...
```

- **Two** methods: `__init__` builds, `forward` computes
- `nn.Module` is the base class: **we don't have to write anything else**

What `nn.Module` gives us for free

- `model(X)`: runs our `forward`. We *write* `forward`, but we *call* the model
- `model.parameters()`: every trainable number, ready to hand to an optimizer
- `model.to(device)`: move everything to the GPU in one call
- `model.state_dict()`: save and load the parameters
- `model.train()` / `model.eval()`: switch modes

> **Note**
>
> None of this is in our code. We inherit it by writing `nn.Module` in the class line and calling `super().__init__()`.

Three rules for a model class

1. **Call `super().__init__()` first**, before assigning any parameters or sub-models
2. **Register what training must change**: wrap raw tensors in `nn.Parameter`, and assign sub-models as attributes
3. **Call `model(X)`**, not `model.forward(X)`: the same `__call__` idea as `line(3)`

- Rule 2 is the one that bites. Next slide.

A Gotcha: We have to tell `torch` about our layers!

```
class Broken(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = [nn.Linear(3, 3), nn.Linear(3, 1)]        # plain list

class Fixed(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([nn.Linear(3, 3), nn.Linear(3, 1)])
```

- A plain Python list is invisible to `nn.Module`: the optimizer would get **zero** parameters and nothing would ever train
- `nn.ModuleList` is a list that `nn.Module` can see into

Reference: the model interface

```python
import torch
from torch import nn

class Model(nn.Module):
    def __init__(self):
        super().__init__()                  # rule 1: always first
        # rule 2: parameters as nn.Parameter, sub-models as attributes
        #         (use nn.ModuleList for a list of sub-models)

    def forward(self, X):
        """Given X, return y_hat."""
        ...

model = Model()
y_hat = model(X)                            # rule 3: call the model, not .forward
model.parameters()                          # everything trainable, found for us
```

Building models

From one line to an MLP

The Big Idea

A model can contain other models, and every one of them has the same interface, so we can stack them like blocks.

Model 1: linear regression, from scratch

```python
class LinearRegression(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.w = nn.Parameter(torch.randn(num_features))
        self.b = nn.Parameter(torch.zeros(1))

    def forward(self, X):
        return X @ self.w + self.b
```

- The same $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ from earlier, now with a home for w and b
- `nn.Parameter` marks a tensor as “training may change this”

Model 2: several models, one model

```
class ParallelLinear(nn.Module):
    def __init__(self, num_features, num_outputs):
        super().__init__()
        self.models = nn.ModuleList(
            [LinearRegression(num_features) for _ in range(num_outputs)]
        )

    def forward(self, X):
        return torch.stack([m(X) for m in self.models], dim=-1)
```

- A model that **contains** models: one `LinearRegression` per output
- This is the earlier “one model per target” idea, now packaged as a single object

Models are blocks

Model 3: layer them into an MLP

```
 1 class MLP(nn.Module):
 2     def __init__(self, num_features, num_hidden, num_outputs):
 3         super().__init__()
 4         self.network = nn.Sequential(
 5             nn.Linear(num_features, num_hidden),
 6             nn.ReLU(),
 7             nn.Linear(num_hidden, num_outputs),
 8         )
 9
10     def forward(self, X):
11         return self.network(X)
12
13 mlp = MLP(3, 4, 2)
14 X = torch.randn(5, 3)                  # 5 examples, 3 features
15 print(mlp(X).shape, sum(p.numel() for p in mlp.parameters()))
```

```
torch.Size([5, 2]) 26
```

- `__init__` **builds** the network once, and `forward` just **runs** it
- `nn.Sequential` is itself a model that runs its pieces in order. Our habit: a straight line of layers goes in `self.network`
- 26 parameters: $3 \cdot 4 + 4$ in the first layer, $4 \cdot 2 + 2$ in the last

> **Note**
>
> When the wiring stops being a straight line (skip connections, coming with CNNs), `forward` does the wiring, and the pieces it connects are sub-models like `self.network`.

Reference: building models (1 of 2)

```
 1 import torch
 2 from torch import nn
 3
 4 class LinearRegression(nn.Module):            # 1. a single linear model
 5     def __init__(self, num_features):
 6         super().__init__()
 7         self.w = nn.Parameter(torch.randn(num_features))
 8         self.b = nn.Parameter(torch.zeros(1))
 9
10     def forward(self, X):
11         return X @ self.w + self.b
12
13
14 class ParallelLinear(nn.Module):              # 2. a model of models
15     def __init__(self, num_features, num_outputs):
16         super().__init__()
```

Reference: building models (2 of 2)

```
class MLP(nn.Module):                                  # 3. layers + a nonlinearity
    def __init__(self, num_features, num_hidden, num_outputs):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(num_features, num_hidden),
            nn.ReLU(),
            nn.Linear(num_hidden, num_outputs),
        )

    def forward(self, X):
        return self.network(X)
```

The training loop

Who does what

From predicting to learning

- A model with random parameters makes bad predictions
- We already know **how** to fix that: score the predictions with a loss, compute the gradient, take a step
- The question now is **which object** does each of those jobs

The Big Idea

Training is a loop where separate objects each do one job, and the loop itself only calls them in the right order.

Who does what

Job	Who does it	In code
Turn inputs into predictions	**the model**	`model(X)`
Score the predictions	**the loss function**	`loss_fn(y_hat, y)`
Compute gradients	**autograd**	`loss.backward()`
Update the parameters	**the optimizer**	`optimizer.step()`
Hand out mini-batches	**the data loader**	`for X, y in loader:`

- The model knows nothing about the optimizer, and the optimizer knows nothing about the model's structure. It only sees `model.parameters()`

The core training loop

```
for epoch in range(num_epochs):    # each epoch = one full pass over the data
    for X, y in loader:            # each mini-batch
        optimizer.zero_grad()      # 1. clear old gradients
        y_hat = model(X)           # 2. forward pass
        loss = loss_fn(y_hat, y)   # 3. score it
        loss.backward()            # 4. gradients, via autograd
        optimizer.step()           # 5. update the parameters
```

- **Outer loop:** epochs. **Inner loop:** mini-batches. Each batch is one SGD step
- This is **the** training loop. Basically the rest of the course uses these five lines, or a variation.

Each line, in the math we know

Line	What it does	Math from earlier
`for X, y in loader:`	hands us one mini-batch	pick a mini-batch
`optimizer.zero_grad()`	clears the last batch's gradients	(bookkeeping, no math)
`y_hat = model(X)`	forward pass	$\hat{y} = Wx + b$, or whatever the model computes
`loss = loss_fn(y_hat, y)`	scores the predictions	$L(\hat{y}, y)$
`loss.backward()`	fills in every parameter's `.grad`	∇L
`optimizer.step()`	moves the parameters	$W \leftarrow W - \eta \nabla_W L$

Breakdown: why `zero_grad()`?

- `loss.backward()` **adds** to each parameter's `.grad`, it does not overwrite it
- Without `zero_grad()`, gradients from the previous batch pile up and our step points the wrong way
- So: clear, then compute, then step. Every batch, in that order

Breakdown: `train()`, `eval()`, and `torch.no_grad()`

After training, we also measure the model on data it hasn't trained on. That uses **two separate switches**:

	What it does	When we use it
`model.train()`	puts layers like dropout and batch norm in training behavior	before the training loop (a new model starts here)
`model.eval()`	puts those same layers in evaluation behavior	before measuring performance
`torch.no_grad()`	stops PyTorch recording gradient information	around any pass where we won't call `.backward()`

Reference: the training loop

```
model = Model()                                          # any nn.Module subclass
loss_fn = nn.MSELoss()                                   # any loss
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)   # any optimizer

for epoch in range(num_epochs):
    model.train()
    for X, y in train_loader:
        optimizer.zero_grad()      # 1. clear old gradients
        y_hat = model(X)           # 2. forward pass
        loss = loss_fn(y_hat, y)   # 3. score it
        loss.backward()            # 4. gradients
        optimizer.step()           # 5. update

    model.eval()
    with torch.no_grad():          # evaluation: predictions only
        for X, y in val_loader:
```

Making it easier on ourselves

Optional packaging on top of the interface

The loop needs three things

- Our loop needs the **model**, the **loss function**, and the **optimizer**, all passed in separately
- They have to **match**: a classifier needs cross-entropy, a regressor needs MSE
  - Nothing stops us from pairing the wrong ones. Keeping them consistent is up to us, every time

Idea: the model already knows what it's for, so let it carry its own loss and optimizer setup

Note

Extending `nn.Module` with a subclass

```
 1 import torch.nn.functional as F
 2
 3 class Regressor(nn.Module):
 4     def __init__(self, lr=0.1):
 5         super().__init__()
 6         self.lr = lr
 7
 8     def loss(self, y_hat, y):
 9         return F.mse_loss(y_hat, y)
10
11     def configure_optimizers(self):
12         return torch.optim.SGD(self.parameters(), lr=self.lr)
13
14     def training_step(self, batch):
15         X, y = batch
16         return self.loss(self(X), y)
```

- `Regressor` **is an** `nn.Module` with three extra methods:
  - `loss` and `configure_optimizers` answer “which loss? which optimizer?”
  - `training_step` does forward pass plus loss for one batch
- It has no `forward` yet: a subclass supplies that

A concrete model, one subclass down

```python
class MLPRegressor(Regressor):
    def __init__(self, num_features, num_hidden, num_outputs, lr=0.1):
        super().__init__(lr)                # parent sets up lr
        self.network = nn.Sequential(
            nn.Linear(num_features, num_hidden),
            nn.ReLU(),
            nn.Linear(num_hidden, num_outputs),
        )

    def forward(self, X):
        return self.network(X)
```

- `MLPRegressor` **is a** `Regressor` **is an** `nn.Module`
- It inherits the loss and optimizer setup, and only writes what is specific to it: the layers and `forward`

A concrete model with branching

```
 1 class BranchingRegressor(Regressor):
 2     def __init__(self, num_features, num_hidden, num_outputs, lr=0.1):
 3         super().__init__(lr)
 4         self.branch_a = nn.Sequential(nn.Linear(num_features, num_hidden), nn.ReLU())
 5         self.branch_b = nn.Sequential(nn.Linear(num_features, num_hidden), nn.ReLU())
 6         self.out = nn.Linear(2 * num_hidden, num_outputs)
 7
 8     def forward(self, X):
 9         both = torch.cat([self.branch_a(X), self.branch_b(X)], dim=-1)
10         return self.out(both)
11
12 model = BranchingRegressor(3, 4, 2)
13 print(model(torch.randn(5, 3)).shape)
```

```
torch.Size([5, 2])
```

Using the packaged model

```
model = MLPRegressor(num_features=3, num_hidden=4, num_outputs=2, lr=0.1)
optimizer = model.configure_optimizers()     # the model picks its optimizer

for epoch in range(num_epochs):
    for batch in loader:
        optimizer.zero_grad()
        loss = model.training_step(batch)    # forward pass + loss, in one call
        loss.backward()
        optimizer.step()
```

- The **same five steps**. No `loss_fn` appears anywhere: the model supplies its own loss and optimizer
- Steps 2 and 3 (forward pass, score it) are folded into `training_step`

Why subclassing pays off

```
class Classifier(Regressor):
    def loss(self, y_hat, y):
        return F.cross_entropy(y_hat, y)
```

- One method overridden, and **the same loop** now trains a classifier
- The loop never changes. The model says what its own loss is

> **Tip**
>
> Reading d2l: its `Module` class is this idea. `training_step`, `loss`, and `configure_optimizers` are packaging, not requirements of the interface.

Reference: packaging with subclasses (1 of 2)

```
import torch.nn.functional as F

class Regressor(nn.Module):                          # adds training conveniences
    def __init__(self, lr=0.1):
        super().__init__()
        self.lr = lr

    def loss(self, y_hat, y):
        return F.mse_loss(y_hat, y)

    def configure_optimizers(self):
        return torch.optim.SGD(self.parameters(), lr=self.lr)

    def training_step(self, batch):
        X, y = batch
        return self.loss(self(X), y)
```

Reference: packaging with subclasses (2 of 2)

```
class MLPRegressor(Regressor):                          # a concrete model
    def __init__(self, num_features, num_hidden, num_outputs, lr=0.1):
        super().__init__(lr)
        self.network = nn.Sequential(
            nn.Linear(num_features, num_hidden),
            nn.ReLU(),
            nn.Linear(num_hidden, num_outputs),
        )

    def forward(self, X):
        return self.network(X)


class Classifier(Regressor):                            # same loop, different loss
    def loss(self, y_hat, y):
        return F.cross_entropy(y_hat, y)
```

Summary

The recipe

Piece	What it is	What we write
Model	an `nn.Module` subclass	`__init__` (build) and `forward` (compute)
Loss	a function of $\hat{y}$ and y	`nn.MSELoss()`, `F.cross_entropy`, …
Optimizer	takes `model.parameters()`	`torch.optim.SGD(...)`, `Adam(...)`
Loop	five lines per batch	zero, forward, loss, backward, step
Packaging *(optional)*	subclass adds `loss` etc.	`training_step`, `configure_optimizers`

Note

Going forward, **every model in this course is an `nn.Module` subclass** with this shape. Convolutions, recurrent layers, and attention are more blocks with the same interface.

Reference: everything on one page

```
import torch
from torch import nn

class Model(nn.Module):
    def __init__(self):
        super().__init__()                          # always first
        self.network = nn.Sequential(               # sub-models / parameters
            nn.Linear(3, 4), nn.ReLU(), nn.Linear(4, 2))

    def forward(self, X):                           # inputs -> predictions
        return self.network(X)

model = Model()
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

```